Secure Key Management Policies in Tamarin

Matteo Busi[1][0000-0002-5557-8139], Riccardo Focardi✉[1][0000-0003-0101-0692], Sana Gul[2,1][0009-0007-4620-2627], and Flaminia Luccio[0000-0002-5409-5039]

[1] Ca' Foscari University, Venice, Italy, {matteo.busi,focardi,luccio}@unive.it
[2] IMT School for Advanced Studies, Lucca, Italy, sana.gul@imtlucca.it

Abstract. Secure key management is critical to cryptographic systems, yet subtle interactions among key operations can compromise otherwise sound cryptographic mechanisms. We present a policy-parametric method for the automated verification of key management APIs in the Tamarin prover. Given a policy specifying which key types may wrap or unwrap other keys, or encrypt and decrypt data, our approach generates the corresponding Tamarin analysis. We show that even simple policies may cause proof search to diverge and develop reusable and policy-specific helper invariants that enable automated verification. We use the resulting analysis to characterize *minimal flawed* and *maximal secure* policies with respect to key secrecy and integrity. Our results expose a duality between these properties, including attacks in which an integrity violation leads to loss of secrecy and vice versa, and scale from exhaustive exploration of small policy spaces to the direct verification of larger concrete policies.

Keywords: Key Management · Formal Verification · Cryptographic APIs

1 Introduction

Cryptographic security ultimately relies on protecting cryptographic keys, making secure key management a critical component of any cryptographic system [21]. A central key-management operation is *key wrapping*: a key is encrypted under another key so that it can be exported or provisioned without being exposed in cleartext; dually, *unwrapping* imports a wrapped key into a protected component. Key-management APIs therefore manipulate both protected keys and ordinary data, and subtle interactions between these two classes of operations can lead to serious vulnerabilities.

The distinction between keys and data is particularly delicate when the same cryptographic primitive is exposed through different API operations. A classical example is the *wrap-then-decrypt* attack on `PKCS#11` [8,20]: if the same key can both wrap keys and decrypt data, an application can wrap a confidential key and subsequently decrypt the resulting ciphertext as ordinary data, exposing the key in cleartext. Mitigations such as `wrap_with_trusted` can prevent some such attacks, but require carefully managed trusted keys and users [13]. This

motivates policy-based approaches that explicitly constrain how keys of different roles may be used.

We study the automated verification of key-management policies using the Tamarin prover [19,2]. We adopt the policy language of [12,6], in which key types determine which other key types may be wrapped or unwrapped and which keys may encrypt or decrypt data.

Previous work [4] showed that fixed, manually instantiated policies can be analyzed in Tamarin. However, that approach did not provide a systematic way to generate and verify arbitrary policies or to characterize entire policy spaces. The present work addresses this limitation through a *policy-parametric* analysis. Given a policy, our `PolicyCompiler` tool generates the corresponding Tamarin artifacts, including policy facts, key-creation rules, secrecy and integrity properties, sanity lemmas, and policy-specific helper invariants required for convergence. This enables systematic analysis of policy spaces rather than verification of individual hand-written instances. Summarizing, our contributions are:

- We develop `PolicyCompiler`, which automatically instantiates a generic Tamarin model from a finite key-management policy and generates the associated security properties and policy-dependent proof artifacts.
- We identify the sources of non-termination in the generated analyses and develop reusable and policy-specific invariants that guide Tamarin's proof search while remaining independently verified proof obligations.
- We derive complete characterizations of key secrecy and integrity for the one-key policy space in terms of *minimal flawed* and *maximal secure* policies, exhaustively analyze all 4096 two-key policies, and identify characteristic secrecy/integrity attacks and their duality.

The Tamarin model and accompanying scripts are available in the anonymized supplementary artifact [1].

2 Background and Related Work

2.1 Tamarin and Threat Model

Tamarin is a symbolic tool for security-protocol analysis based on multiset rewriting [19]. A rule has the form

$$[\ p_1, \ldots, p_m\ ]\ \texttt{--[}\ a_1, \ldots, a_k\ \texttt{]->}\ [\ c_1, \ldots, c_n\ ]$$

where p_i, a_i, and c_i are premise, action, and conclusion facts, respectively. Premise and conclusion facts describe the protocol state, while action facts record events on an execution trace. A rule fires when its premise facts are available; firing consumes the non-persistent premises, adds the conclusion facts to the state, and records the action facts on the trace. The built-in facts `In(m)` and `Out(m)` model communication with a Dolev–Yao adversary, while `Fr(n)` generates fresh values. Security properties are first-order temporal formulas over action facts. Tamarin either proves a property, produces a counterexample trace, or may fail to terminate.

Threat model. We consider a Dolev–Yao adversary interacting with a trusted cryptographic API. The adversary controls API inputs and network-visible messages, may invoke all operations permitted by the policy, and observes their public outputs, but cannot directly access internally stored key facts. The trusted component stores keys and enforces the policy. Our core model considers a single API instance; multi-device deployments are outside its scope. The model also excludes side-channel leakage, such as power, timing, or electromagnetic emissions during `wrap`, `unwrap`, or `decrypt`. A policy proved secure admits no attack through the modeled API operations, but this does not guarantee that its implementation resists side-channel or other physical attacks.

Ric: I shortened a bit, to stay in the page limit

Fla: ok

Matteo: ok

2.2 A General Model for Key Management Policies

Key-management policies specify which cryptographic operations keys of different types may perform. We adopt the model of [12], with a finite set of key types

$$\mathcal{K} = \{K_1, K_2, \ldots, K_N\}$$

and a distinguished type D for ordinary data. Let $\mathcal{KD} = \mathcal{K} \cup \{D\}$ and $\mathcal{L} = \{e, d\}$, where e and d denote encryption and decryption, respectively.

A key-management policy is a relation

$$P \subseteq \mathcal{K} \times \mathcal{L} \times \mathcal{KD}.$$

We write

$$K \xrightarrow{l}_P J$$

when $(K, l, J) \in P$. Thus, $K_1 \xrightarrow{e}_P K_2$ states that keys of type K_1 may encrypt, or wrap, keys of type K_2, whereas $K_3 \xrightarrow{d}_P D$ allows keys of type K_3 to decrypt ordinary data. When both encryption and decryption are permitted, we write $K \xrightarrow{e/d}_P J$. Since data cannot itself act as a cryptographic key, the source of every policy edge is always a type in $\mathcal{K}$.

This distinction between operations on keys and operations on data is central: the same underlying cryptographic direction may correspond to key wrapping or ordinary data encryption, and unsafe combinations of these capabilities can compromise key secrecy or integrity.

2.3 Related Work

Prior studies consistently show that developers often misuse cryptographic APIs [17,24]. Our work on secure key management complements this line of research by enforcing formally verified policies that are immune to such misuses. Embedding secure key-management policies at the API level enables automated detection and prevention of key-operation misuse.

Starting with the *wrap-then-decrypt* attack by Clulow [8], significant effort has addressed API attacks on `PKCS#11` and possible mitigations (see, e.g.,

[3,7,8,10,11,13,14,16,18]). However, previous work either focused on specific key configurations, i.e., sets of capabilities that apply uniformly to all keys, or on mechanisms based on the `wrap_with_trusted` attribute, which allows a security officer to designate special `trusted` keys [15]. In contrast, we consider general policies based on key types, which regulate key usage according to the assigned type; key configurations are instances of such policies.

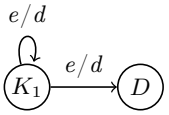
Dax et al. [10] proposed a formally verified mechanism called *authenticated wrapping*, which requires key attributes to be wrapped together with their corresponding key so that they remain unchanged upon import and export. Although proposed to OASIS [22], related authenticated-wrapping mechanisms were only partially reflected in later `PKCS#11` revisions. A common challenge for proposed fixes is deployment within existing `PKCS#11` interfaces and interoperability constraints. Motivated by this restriction, a configuration for cloud HSMs that requires no API changes was proposed in recent work [13], though it assumes trust in special users, called *key managers*, regarding key attribute handling. The challenge of incorporating a general and flexible key-management policy into `PKCS#11` remains unresolved. In this work, we contribute to this problem by investigating automated methods for verifying the security of key-management policies, with the goal of eventually providing a concrete proposal suitable for integration into existing standard APIs. Our work complements previous approaches and focuses on systematically characterizing the inherent properties of key-management policies.

Recent work has studied key-management policies using Strand Spaces [23]. A sound static analysis for detecting whether a key of a given initial type may be disclosed was introduced in [12] and later mechanized and refined in Rocq [6,5]. We use these closure analyses both as a point of comparison for our Tamarin results (Section 5.4) and to derive the reachability information used by `PolicyCompiler` to generate policy-specific helper invariants (Section 4.4).

The closest related work is [4], which showed that selected, manually instantiated key-management policies can be analyzed directly in Tamarin. The present work uses the same policy model and a closely related core Tamarin encoding, but makes the analysis policy-parametric: given a policy, `PolicyCompiler` automatically generates the corresponding policy facts, key-creation rules, security properties, sanity lemmas, and policy-specific proof invariants. This enables systematic exploration of bounded policy spaces as well as direct verification of larger concrete policies, while additionally considering key integrity.

Ric: I use Our since this is the camera ready ok? I've checked this and addend in the README.md file the reference to the exact models

Our earlier work [4] analyzed five manually instantiated key-management policies with up to four key types. Each case study required policy-specific Tamarin rules and a source lemma developed in part from Tamarin's counterexamples. The present work uses the same policy model and a closely related core encoding, but makes the analysis policy-parametric: given a policy, `PolicyCompiler` [1] generates its policy facts, key-creation rules, security properties, sanity lemmas, and proof invariants. This enables us to analyze all 16 one-key policies and to characterize all 4096 two-key configurations using ver-




```
rule P1:
[ ] -->
[!Policy('K1', 'enc', 'K1'),
 !Policy('K1', 'dec', 'K1'),
 !Policy('K1', 'enc', 'D'),
 !Policy('K1', 'dec', 'D')]
```

Fig. 1: The policy $P1$ and its implementation in Tamarin.

ified boundary policies and monotonicity, as well as to verify selected larger policies directly.

All five policies studied in our earlier work are covered by `PolicyCompiler`, and their automatically generated Tamarin models are included in the supplementary artifact [1]. None requires a policy construct outside the tool's supported model. The present analysis also checks key integrity in addition to secrecy. Conversely, its generated secrecy properties do not yet capture the conditional dependencies examined in our earlier work, such as the compromise of keys of one type following the leakage of a key of another type. Automating those conditional properties remains future work.

Ric: We forgot this is not done yet....

3 Methodology and Formal Modeling

In this section, we present our methodology for verifying the security of key-management APIs with the Tamarin Prover and for systematically analyzing classes of key-management policies.

3.1 A Core Key Management API

Our model captures five core cryptographic operations that reflect a typical key-management scenario: (i) `create`, the process of generating a fresh key; (ii) `wrap`, the encryption of one cryptographic key for secure storage or transmission; (iii) `unwrap`, the reverse of wrapping, i.e., decrypting a wrapped key (typically within a trusted component like a TEE, HSM, or smartcard) to recover the plaintext key for subsequent use without exposing it outside the protected environment; (iv) `encrypt`, the process of encrypting data; and (v) `decrypt`, the process of decrypting data.

We write `W`, `U`, `E`, and `D` for the four policy-controlled API operations `wrap`, `unwrap`, `encrypt`, and `decrypt`, respectively. At the policy level they correspond to

$$\texttt{W}: (K, e, K') \qquad \texttt{U}: (K, d, K')$$
$$\texttt{E}: (K, e, D) \qquad \texttt{D}: (K, d, D).$$

Thus, e/d denotes the underlying cryptographic direction, whereas `W`/`U`/`E`/`D` denote API-level operations.

Table 1: The core API in Tamarin, parametrized by the policy.

```
rule CreateKey_K1:
  [ Fr(k) ]
--[ CreateKey(k,'K1') ]->
  [ !Key(k,'K1') ]

rule Wrap:
  [ !Policy(K2, 'enc', K1),
    !Key(k,K1), !Key(j,K2) ]
--[ Wrap(k,j,K1,K2), ND(K1) ]->
  [ Out(senc(k,j)) ]

rule Unwrap:
  [ !Policy(K2, 'dec', K1),
    !Key(j,K2), In(senc(k,j)) ]
--[ Unwrap(k,j,K1,K2), ND(K1) ]->
  [ !Key(k,K1) ]
```

```
rule Encrypt:
  [ !Policy(K0, 'enc', 'D'),
    !Key(k,K0), In(m) ]
--[ Encrypt(m,k,K0) ]->
  [ Out(senc(m,k)) ]

rule Decrypt:
  [ !Policy(K0, 'dec', 'D'),
    !Key(k,K0), In(senc(m,k)) ]
--[ Decrypt(m,k,K0) ]->
  [ Out(m) ]

restriction NonData:
  "All x #i. ND(x) @i ⟹
                not (x = 'D')"
```

3.2 Policy-Based Key Management in Tamarin

A key-management policy, as defined in Section 2.2, is a set of triples (K_1, l, J), where $l \in \{e, d\}$ and J is either a key type or the data type D. We write $K_1 \xrightarrow{l}_P J$ to state that $(K_1, l, J) \in P$, i.e., that a key of type K_1 may perform operation l on an object of type J. We encode each such triple in Tamarin as an instance of the persistent fact `Policy` with three public-constant parameters representing K_1, l, and J. The policy name itself is not encoded in the fact, since each generated Tamarin theory represents a single policy. For example, $K_1 \xrightarrow{e}_P K_2$ is encoded as `!Policy('K1', 'enc', 'K2')`.

Example 1. Consider the complete, maximally permissive policy $P1$ for a single key type K_1, shown on the left side of Fig. 1. Recall that the notation e/d on a single arrow is shorthand for two arrows, one labeled e and the other d (see Section 2.2). The corresponding Tamarin encoding is shown on the right side of Fig. 1. Each arrow in $P1$ is encoded as a separate occurrence of the persistent fact `Policy`. The Tamarin rule `P1` unconditionally generates these facts, which are then used to regulate the cryptographic operations, as discussed next.

The generic API rules are shown in Table 1. Fresh keys are stored internally as persistent `Key` facts. The remaining rules are enabled by persistent `Policy` facts: `wrap` and `encrypt` produce symbolic ciphertexts, whereas `unwrap` imports the recovered value as an internal key and `decrypt` exposes it as plaintext. The `ND` restriction distinguishes key-to-key operations from operations on data.

The `unwrap` and `decrypt` rules pattern-match on terms of the form `senc(m,k)`, so they succeed only on well-formed ciphertexts under the expected key. This models a decryption interface with ciphertext-integrity checks, which a concrete implementation must provide through authenticated encryption or an equivalent mechanism. We make this assumption to isolate attacks caused by the policy's

permitted operations from attacks caused by weak cryptography: our analysis asks whether a policy preserves key secrecy and integrity even when ciphertext integrity is enforced.

Ric: Makes sense? We don't want policies to be bypassed...

Only key-creation rules depend directly on the set of key types: one `CreateKey` rule is generated for each type. The remaining API rules are shared by all policies and are enabled by the corresponding `Policy` facts. Keys enter the device only through fresh generation or `unwrap`; the model has no direct cleartext key-import rule. This does not imply that every imported key has trusted provenance. If a policy permits attacker-chosen data to be encrypted and the resulting ciphertext to be unwrapped as a key, the attacker can effectively import chosen key material into the target type. Our key-integrity lemmas detect this policy-dependent behavior.

Ric: This answers the other reviewer's comment

To automate policy instantiation, we developed `PolicyCompiler`, a tool that takes a finite key-management policy as input and generates the policy-specific Tamarin artifacts required for its analysis. Thus, the same core theory is reused across policy instances, while policy-dependent rules and lemmas are generated automatically by `PolicyCompiler`. We describe the generation of security properties and policy-specific helper invariants in Section 4.4.

The model permits the reuse of keys across rules, including the possibility of self-encryption (e.g., terms of the form `senc(k,k)`), and therefore does not exclude *key cycles*. As in standard symbolic models, encryption is treated as a perfect primitive, and computational soundness is not considered in this paper. Nevertheless, due to the presence of key cycles, our results should not be expected to provide computational guarantees in settings where key cycles are known to be problematic.

4 Security Analysis

For our analysis, we consider four classes of lemmas: auxiliary source lemmas (Section 4.1), key-secrecy lemmas (Section 4.2), key-integrity lemmas (Section 4.3), and existential sanity lemmas for checking executability, which are reported in the artifact [1].

4.1 Source Lemmas

Rules that consume ciphertexts, namely `decrypt` and `unwrap`, require Tamarin to reason backwards about the possible origin of the recovered term. Without additional guidance, these origins may remain ambiguous, resulting in *partial deconstructions* and non-terminating proof search. We initially experimented with Tamarin's automatic source-lemma generation [9], but it diverged on policies containing both `wrap` and `unwrap`. We therefore use the generic source lemma shown in Table 2.

Lemma `SourcesDU` over-approximates the possible origins of values recovered by `decrypt` and `unwrap`: such a value must either have been generated internally

Table 2: Source lemma.

```
lemma SourcesDU [sources]:
"                                          &
(All m k K0 #j.                            (All m k K1 K2 #j.
  Decrypt(m,k,K0) @ #j ⟹                     Unwrap(m,k,K1,K2) @ #j ⟹
    (Ex K1 #i. CreateKey(m,K1) @ #i             (Ex K1 #i. CreateKey(m,K1) @ #i
               & #i < #j)                                  & #i < #j)
    |                                          |
    (Ex #i. !KU(m) @ #i & #i < #j))            (Ex #i. !KU(m) @ #i & #i < #j))"
```

Table 3: Key secrecy lemma for type `K1`.

```
lemma Secrecy_K1:
  "All k #j. CreateKey(k,'K1') @ #j ⟹
    not(Ex #i. K(k) @ #i)"
```

as a key or have been known to the adversary beforehand. The `[sources]` annotation makes this invariant available during Tamarin's source precomputation.

Importantly, source lemmas are not assumptions: they remain proof obligations, and our analysis accepts subsequent secrecy or integrity results only when the source lemma has itself been successfully proved by Tamarin. Here, `K(m)` denotes that the adversary can derive the term m, whereas `!KU(m)` is Tamarin's internal persistent attacker-knowledge fact used during proof search. We use `!KU` in this auxiliary lemma because replacing it with the user-facing `K` causes proof search to diverge; final secrecy properties still use `K`.

4.2 Key Secrecy

Key secrecy requires that every key generated by the device remain unknown to the adversary. For readability, we first consider a single key type `K1`; multiple key types are discussed in Section 4.4. Lemma `Secrecy_K1` in Table 3 formalizes this property by requiring that a key recorded by a `CreateKey` event is never learned by the adversary.

Direct verification of this property diverges for some otherwise simple policies, including `W`, `WU`, and `WE`. The most interesting case is `WU`, for which proving secrecy requires information about the origin of imported keys, while proving integrity requires knowing that device-generated keys cannot first leak and later be reused by the adversary. For the `WU` policy, we use the combined invariant `General_IS` shown in Table 4. Its first conjunct requires values imported through `unwrap` to have been previously generated as `K1` keys, while its second conjunct prevents generated `K1` keys from becoming attacker knowledge. The two conditions provide the strengthening needed by Tamarin's backward search when secrecy and integrity depend on one another.

The `[reuse]` annotation makes the proved invariant available to subsequent security proofs. Importantly, `General_IS` is never used as an unproved assump-

Table 4: Policy-specific helper invariant for the one-key WU policy.

```
lemma General_IS [reuse]:
"(All k1 k2 K1 #j.
  Unwrap(k1,k2,'K1',K1) @ #j ⟹
    (Ex #i. CreateKey(k1,'K1') @ #i & #i < #j))
&
(All k #i #j.
  CreateKey(k,'K1') @ #j & !KU(k) @ #i ⟹ F)"
```

Table 5: Key integrity lemma.

```
rule IsKey:
  [ !Key(k,K0) ] --[ IsKey(k,K0) ]-> [ ]

lemma Integrity_K1:
"All k #j. IsKey(k,'K1') @ #j ⟹
  (Ex K2 #i. CreateKey(k, K2) @ #i & #i < #j)"
```

tion: if it is false or cannot be established for a policy, security results whose proof chain depends on it are not accepted.

4.3 Key Integrity

Key integrity is dual to secrecy: while secrecy prevents device-generated key material from becoming attacker knowledge, integrity prevents attacker-controlled values from becoming internal keys. We observe the presence of an internal key through the action fact `IsKey(k,K0)` and define integrity as in Table 5: every internal key value must originate from some previous `CreateKey` event.

This is an *origin-integrity* property. A value present as type `K1` must have been generated by the device, but it need not have been generated originally with type `K1`. Preserving the original key type would constitute a distinct, stronger property.

As for secrecy, direct verification diverges for some policies, namely `U`, `WU`, and `UD`. These cases are handled by the combined invariant `General_IS` introduced in Section 4.2, reflecting the mutual dependence between secrecy and integrity during proof search. Interestingly, this dependence is itself dual: the problematic secrecy cases include `W` and `WE`, whereas the corresponding integrity cases include `U` and `UD`.

4.4 Policy-Parametric Analysis

Given a finite policy P, `PolicyCompiler` generates the policy-specific artifacts required by the generic theory. For every key type, it generates a key-creation rule and the corresponding secrecy and integrity properties; it additionally generates

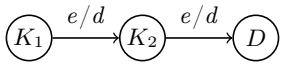


Fig. 2: Secure hierarchical policy with two key types.

Table 6: Policy-specific helper invariant generated for the policy of Figure 2.

```
lemma General_IS [reuse]:
"(All k1 k2 K1 #j.
  Unwrap(k1,k2,'K1',K1) @ #j ⟹ F )
&
(All k1 k2 K1 #j.
  Unwrap(k1,k2,'K2',K1) @ #j ⟹
    (Ex #i. CreateKey(k1,'K2') @ #i & #i < #j)
)
&
(All k #i #j.
  CreateKey(k,'K1') @ #j & !KU(k) @ #i ⟹ F)
&
(All k #i #j.
  CreateKey(k,'K2') @ #j & !KU(k) @ #i ⟹ F)"
```

sanity lemmas and a policy-specific `General_IS` invariant. The core cryptographic rules and the generic source lemma remain unchanged.

The policy-specific invariant is obtained using the closure analyses of [12,6], which over-approximate the key types reachable from a given initial type. This information allows `PolicyCompiler` to constrain relevant `unwrap` events and, where applicable, key disclosure, providing Tamarin with the invariants needed for automated proof search.

Example 2. Consider the hierarchical two-key policy in Fig. 2. For this policy, `PolicyCompiler` generates a `General_IS` invariant that forbids imports as type K_1, requires every value imported as K_2 to originate from device-generated K_2 material, and preserves secrecy for both key types. The resulting invariant is shown in Table 6. Its precision is essential for automation: weakening the constraints on `unwrap` causes the analysis to diverge.

5 Results and Findings

We first examine all 16 policies over a single key type (Section 5.1), which provide a compact setting for identifying the main secrecy and integrity patterns. We then characterize security in terms of *minimal flawed* and *maximal secure* policies (Section 5.2), analyze minimal attacks (Section 5.3), and compare our results with related work (Section 5.4). Finally, we characterize the complete two-key policy space and discuss representative cross-type attacks (Section 5.5); the full verification results are available in the supplementary artifact [1].

Table 7: Verification results for all 16 single-key policies.

			Secrecy	Integrity	Sanity			
Policy	SourcesDU	General_IS	K1	K1	W	U	E	D
-	✓	✓	✓	✓				
W	✓	✓	✓	✓	✓			
U	✓	✓	✓	✓		✗		
E	✓	✓	✓	✓			✓	
D	✓	✓	✓	✓				✗
WU	✓	✓	✓	✓	✓	✓		
WE	✓	✓	✓	✓	✓		✓	
WD	✓		✗	✓	✓			✓
UE	✓	✓	✓	✗		✓	✓	
UD	✓	✓	✓	✓		✗		✗
ED	✓	✓	✓	✓			✓	✓
WUE	✓		✗	✗	✓	✓	✓	
WUD	✓		✗	✗	✓	✓		✓
WED	✓		✗	✓	✓		✓	✓
UED	✓	✓	✓	✗		✓	✓	✓
WUED	✓		✗	✗	✓	✓	✓	✓

All experiments were conducted using Tamarin 1.10.0, Maude 2.7.1, and Graphviz 14.1.2 on a MacBook Pro M4 with 24 GB of RAM. Generating and analyzing all single-key policies takes approximately 15 seconds, while the complete two-key analysis takes approximately 8 minutes and 30 seconds.

5.1 Exhaustive Analysis of Policies with a Single Key Type

With the aid of source and helper lemmas, verification terminates for all 16 single-key policies, as summarized in Table 7.

Individual operations are secure. Each individual operation, `W`, `U`, `E`, or `D`, preserves both secrecy and integrity, providing a useful sanity check.

Critical combinations. Two minimal combinations already compromise security. Policy `WD` violates secrecy through the classical wrap-then-decrypt attack [8], whereas `UE` violates integrity by allowing attacker-controlled key material to be encrypted and subsequently imported through `unwrap`. The latter sequence appears in [11] as part of a secrecy attack, but its role as an integrity violation has not been highlighted. Any policy containing `WD` therefore violates secrecy, while any policy containing `UE` violates integrity.

Interplay between secrecy and integrity. The two properties also interact. Policy `WUE` violates secrecy without containing `WD`: the `UE` integrity violation first introduces attacker-controlled key material, which is then used to compromise secrecy. Dually, `WUD` violates integrity because the `WD` secrecy attack first reveals key material that can subsequently be used to construct a value accepted by `unwrap`. Thus, a failure of either property may enable a failure of the other.

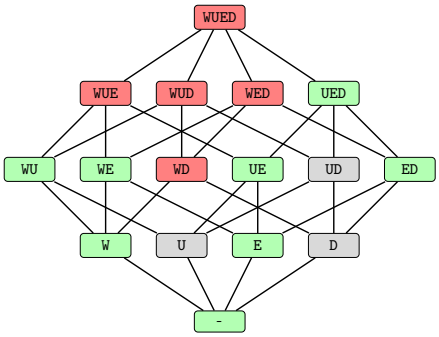


Fig. 3: Secrecy lattice.

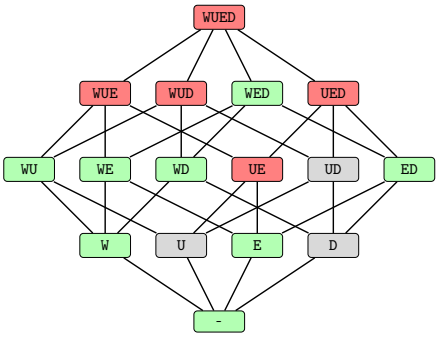


Fig. 4: Integrity lattice.

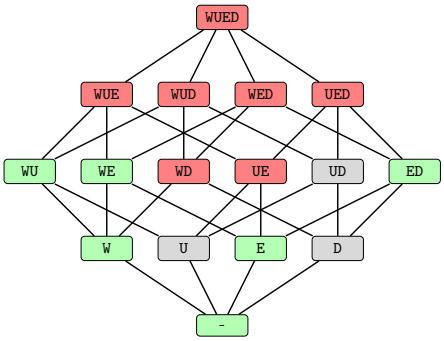


Fig. 5: Secrecy and integrity lattice.

Void policies. The policies `U`, `D`, and `UD` are void in our single-device model: they only consume ciphertexts, while no enabled operation can produce them. They are therefore trace-equivalent to the empty policy. Such policies may become meaningful in multi-device deployments, but their single-device security does not imply security when complementary capabilities are combined.

5.2 Logical Characterization of Secrecy and Integrity

Both secrecy and integrity are safety properties and are monotone with respect to policy inclusion. If $P \subseteq P'$, every trace of P is also a trace of P', since P' only enables additional API operations. Hence, if P admits an attack trace, every policy containing P admits the same trace. This motivates ordering policies by set inclusion and characterizing security through *minimal flawed* and *maximal secure* policies.

For secrecy, the minimal flawed policies are `WD` and `WUE`; for integrity, they are `UE` and `WUD`. Thus:

$$P \textit{ satisfies secrecy} \;\Leftrightarrow\; \neg(\texttt{WD} \subseteq P \vee \texttt{WUE} \subseteq P),$$
$$P \textit{ satisfies integrity} \;\Leftrightarrow\; \neg(\texttt{UE} \subseteq P \vee \texttt{WUD} \subseteq P).$$

Table 8: Comparison with prior work [12,6].

Policy	Secrecy	Integrity	Closures (Secrecy)		Closures (Integrity)	
	K1	K1	Original	Improved	Original	Improved
-	✓	✓	✓	✓	✓	✓
W	✓	✓	✓	✓	✓	✓
U	✓	✓	✓	✓	✓	✓
E	✓	✓	✓	✓	✓	✓
D	✓	✓	✓	✓	✓	✓
WU	✓	✓	✓	✓	✓	✓
WE	✓	✓	✓	✓	✓	✓
WD	✗	✓	✳	✳	✓	✓
UE	✓	✗	✳	✓	✳	✳
UD	✓	✓	✓	✓	✓	✓
ED	✓	✓	✓	✓	✓	✓
WUE	✗	✗	✳	✳	✳	✳
WUD	✗	✗	✳	✳	✳	✳
WED	✗	✓	✳	✳	✓	✓
UED	✓	✗	✳	✓	✳	✳
WUED	✗	✗	✳	✳	✳	✳

Dually, the maximal secure policies are UED, WU, and WE for secrecy, and WED, WU, and UD for integrity. Consequently:

$$P \textit{ satisfies secrecy} \;\Leftrightarrow\; P \subseteq \texttt{UED} \vee P \subseteq \texttt{WU} \vee P \subseteq \texttt{WE},$$
$$P \textit{ satisfies integrity} \;\Leftrightarrow\; P \subseteq \texttt{WED} \vee P \subseteq \texttt{WU} \vee P \subseteq \texttt{UD}.$$

Figs. 3 and 4 show the secrecy and integrity lattices, respectively. The combined lattice, shown in Fig. 5, highlights policies satisfying both properties. Void policies are marked separately, since UD is trace-equivalent to the empty policy in our single-device model.

When both properties are required, the minimal flawed policies reduce to WD and UE, while the maximal secure policies are WU, WE, UD, and ED. Writing P *is secure* when both secrecy and integrity hold, we obtain:

$$P \textit{ is secure} \;\Leftrightarrow\; \neg(\texttt{WD} \subseteq P \vee \texttt{UE} \subseteq P),$$
$$P \textit{ is secure} \;\Leftrightarrow\; P \subseteq \texttt{WU} \vee P \subseteq \texttt{WE} \vee P \subseteq \texttt{UD} \vee P \subseteq \texttt{ED}.$$

5.3 Minimal Attacks

We inspect the counterexamples produced by Tamarin for the minimal flawed policies, which are representative of their more permissive supersets. Tamarin attack graphs for the representative counterexamples discussed below are included in the supplementary artifact [1]. In the traces below, `~n` denotes a freshly generated key and `k` an attacker-controlled value.

Wrap-then-decrypt. For WD, Tamarin recovers the classical secrecy attack [8], in which a freshly generated key is wrapped and the resulting ciphertext is decrypted as data:

```
CreateKey(~n,'K1'); CreateKey(~n1,'K1');
Wrap(~n,~n1,'K1','K1'); Decrypt(~n,~n1,'K1')
```

Encrypt-then-unwrap. For `UE`, attacker-controlled material is encrypted and then imported as a key, violating integrity:

```
CreateKey(~n,'K1'); Encrypt(k,~n,'K1'); Unwrap(k,~n,'K1','K1')
```

Secrecy through an integrity violation. For `WUE`, Tamarin initially produces a non-minimal counterexample. Restricting the secrecy lemma to traces containing a single `wrap` operation yields the shorter attack below, corresponding to the attack of [11]:

```
CreateKey(~n,'K1'); CreateKey(~n.1,'K1'); Encrypt(j.1,~n.1,'K1');
Unwrap(j.1,~n.1,'K1','K1'); Wrap(~n,j.1,'K1','K1')
```

Integrity through a secrecy violation. For `WUD`, Tamarin finds the dual attack: a wrap-then-decrypt sequence first reveals a device-generated key, which the attacker then uses to construct a ciphertext importing `k`:

```
CreateKey(~n,'K1'); CreateKey(~n1,'K1');
Wrap(~n,~n1,'K1','K1'); Decrypt(~n,~n1,'K1')
The attacker learns ~n and forges senc(k,~n)
Unwrap(k,~n,'K1','K1')
```

To our knowledge, this integrity attack has not previously been reported.

5.4 Comparison with Previous Work

We compare our results with the policy analyses introduced in [12] and later refined and mechanized in Rocq [6]. These analyses compute a closure of the policy that over-approximates the key types reachable from a given initial type. For secrecy, their soundness results guarantee that a key of type `K1` remains secret whenever D is not reachable from `K1`.

We implemented both the original and refined closures and applied them to the single-key policies considered here. Table 8 compares their secrecy results with those obtained in Tamarin. The original closure is less precise for `UE` and `UED`, which it marks as potentially insecure although Tamarin proves them secure. The refined closure agrees with Tamarin on all single-key policies in our experiments. This agreement provides empirical evidence of completeness for this policy space, but does not constitute a general completeness proof.

We also investigate a dual use of the closure for key integrity. While the existing analyses check whether a key type can reach the public type D, we test whether D can reach `K1` as a candidate integrity criterion. For all single-key policies considered here, both closure analyses agree with the integrity results obtained in Tamarin. We therefore conjecture that this dual criterion provides a sound and complete characterization of integrity, but a formal proof remains future work.

Ric: I've basically rewritten this section. The numbers are verified by a new script that we will include with the artifact.

5.5 Policies with Two Key Types

With two key types, the policy space contains 4096 configurations. We explore it using *minimal flawed* and *maximal secure* policies, together with the policy-specific helper generation described in Section 4.4. The invariants are combined with proof-search heuristics that prioritize `Key` goals, omit the helper when its secrecy conditions do not hold, and fall back to Tamarin's `S` heuristic for cases that do not converge otherwise. We compute candidate bases using the refined closure analysis of [6]. Since its completeness has not been established, we check that the bases classify all 4096 configurations without conflicts and verify every boundary result in Tamarin. Monotonicity establishes the remaining results.

For $\mathtt{K}_1$, we obtain 25 minimal flawed and 16 maximal secure policies for secrecy, and 23 and 14, respectively, for integrity. These 78 boundary checks involve 69 distinct configurations; the other 4027 require no direct Tamarin run. We also verified the corresponding bases for $\mathtt{K}_2$ as a symmetry check, obtaining the $\mathtt{K}_1$ results with the types exchanged. The 156 Tamarin runs all terminated with the selected heuristics; two configurations, each checked for both types, required the `S` heuristic. The complete bases, prover results, and heuristic choices appear in the supplementary artifact [1].

By monotonicity, 1397 policies preserve $\mathtt{K}_1$ secrecy, 1379 its integrity, and 749 both; 499 preserve both properties for both key types. In 1555 configurations, the subpolicy restricted to $\mathtt{K}_1$ preserves both properties, but the full policy violates at least one for $\mathtt{K}_1$. Each contains a cross-type wrap or unwrap permission, showing that analyzing the types in isolation misses these attacks. Requiring secrecy and integrity gives 22 minimal flawed policies for $\mathtt{K}_1$ alone, but only 16 for both types: larger attacks on one type may contain smaller attacks on the other. The following criterion excludes these 16 patterns.

Write $\mathtt{W}_{\mathtt{K_i},\mathtt{K_j}}$ and $\mathtt{U}_{\mathtt{K_i},\mathtt{K_j}}$ for permissions in which a key of type $\mathtt{K_i}$ acts on a key of type $\mathtt{K_j}$, and $\mathtt{E}_{\mathtt{K_i}}, \mathtt{D}_{\mathtt{K_i}}$ for operations on data. Let $i, j \in \{1, 2\}$. For $i \neq j$, define

$$\mathcal{B}_{i,j} = \left\{ \mathtt{W}_{\mathtt{K_j},\mathtt{K_j}}\, \mathtt{U}_{\mathtt{K_j},\mathtt{K_i}}, \mathtt{W}_{\mathtt{K_j},\mathtt{K_i}}\, \mathtt{U}_{\mathtt{K_j},\mathtt{K_j}} \right\}.$$

Thus, secrecy and integrity hold for both key types exactly when

$$\neg\Big(\bigvee_{i,j}\big(\mathtt{W}_{\mathtt{K_i},\mathtt{K_j}}\,\mathtt{D}_{\mathtt{K_i}} \subseteq P \vee \mathtt{U}_{\mathtt{K_i},\mathtt{K_j}}\,\mathtt{E}_{\mathtt{K_i}} \subseteq P\big) \;\vee \bigvee_{i\neq j}\bigvee_{B\in\mathcal{B}_{i,j}}\big(B\,\mathtt{D}_{\mathtt{K_i}} \subseteq P \vee B\,\mathtt{E}_{\mathtt{K_i}} \subseteq P\big)\Big).$$

The first disjunction captures eight direct two-permission patterns; the second captures eight additional three-permission patterns involving both key types. With a single key type, the criterion reduces to $\neg(\mathtt{WD} \subseteq P \vee \mathtt{UE} \subseteq P)$, as in Section 5.2.

For example, $\mathtt{W}_{\mathtt{K_1},\mathtt{K_2}}\,\mathtt{U}_{\mathtt{K_1},\mathtt{K_1}}\,\mathtt{D}_{\mathtt{K_2}}$ is a minimal pattern captured by the second disjunction. Under this policy, a $\mathtt{K}_2$ key can be re-imported as $\mathtt{K}_1$, making the same key material available under both types. It then wraps another $\mathtt{K}_2$ key as $\mathtt{K}_1$ and decrypts the ciphertext as $\mathtt{K}_2$, revealing that key.

Maximal policies preserving $\mathtt{K}_1$ secrecy can contain all four policy-controlled operations. For example,

$$\mathtt{W}_{\mathtt{K_2},\mathtt{K_2}}\,\mathtt{W}_{\mathtt{K_1},\mathtt{K_2}}\,\mathtt{U}_{\mathtt{K_2},\mathtt{K_2}}\,\mathtt{U}_{\mathtt{K_2},\mathtt{K_1}}\,\mathtt{U}_{\mathtt{K_1},\mathtt{K_2}}\,\mathtt{U}_{\mathtt{K_1},\mathtt{K_1}}\,\mathtt{E}_{\mathtt{K_2}}\,\mathtt{E}_{\mathtt{K_1}}\,\mathtt{D}_{\mathtt{K_2}}\,\mathtt{D}_{\mathtt{K_1}}$$

preserves secrecy of $\mathtt{K}_1$ because $\mathtt{K}_1$ keys are never wrapped. It does not preserve $\mathtt{K}_1$ integrity, since $\mathtt{U}_{\mathtt{K}_1,\mathtt{K}_1}\,\mathtt{E}_{\mathtt{K}_1} \subseteq P$.

Exhaustive exploration is practical only for small bounded policy spaces, whose size grows exponentially with the number of key types. The workflow can also generate and analyze a specific finite policy without enumerating the surrounding space. We used it to verify the policies studied in our earlier work [4], including those of [12], with up to four key types, as well as a hierarchical family with two to four key types. Verification times for these larger examples ranged from approximately 0.3 to 30 seconds. Thus, the same workflow supports both exhaustive exploration of small policy spaces and direct verification of selected larger policies.

6 Conclusion

We presented a policy-parametric workflow for the automated verification of key-management APIs in Tamarin. Given a finite policy, `PolicyCompiler` generates the corresponding Tamarin artifacts, security properties, sanity checks, and policy-specific invariants needed for automated proof search.

Using this workflow, we exhaustively analyzed all one-key policies and all 4096 two-key policies, and directly verified selected larger policies without enumerating their surrounding policy spaces. The analysis yields concise characterizations in terms of *minimal flawed* and *maximal secure* policies and exposes a close interaction between key secrecy and integrity: violations of either property can enable attacks on the other.

A central challenge is proof automation. Generic source lemmas resolve ambiguous origins of decrypted terms, while policy-specific invariants constrain key import and disclosure sufficiently for Tamarin to converge. These invariants remain independently verified proof obligations rather than assumptions.

Finally, our comparison with closure-based analyses shows experimental agreement with the refined secrecy analysis on all single-key policies considered here and motivates a dual reachability criterion for integrity. Establishing the general completeness of the refined secrecy characterization and proving the conjectured integrity criterion remain directions for future work. Tamarin's support for mutable state makes extending our approach to dynamic permissions a natural direction for future work. A further direction is relaxing the assumptions of Section 3.2.

Ric: What if we just say this regarding dynamic permissions?

Ric: I've removed appendix as I believe it is not allowed in the camera ready

Ric: What does this mean?

Sana: dynamic policies, such as keys valid for a limited time or a bounded number of uses

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Supplementary material. https://osf.io/5nwxr/overview?view_only=8e2d11733b8b454299c61187931be675 (2026), Anonymized supplementary artifact for Secure Key Management Policies in Tamarin

2. Basin, D., Cremers, C., Dreier, J., Sasse, R.: Tamarin: Verification of Large-Scale, Real-World, Cryptographic Protocols. IEEE Security & Privacy **20**(3), 24–32 (2022). https://doi.org/10.1109/MSEC.2022.3153648
3. Bortolozzo, M., Centenaro, M., Focardi, R., Steel, G.: Attacking and fixing PKCS#11 security tokens. In: Proceedings of the 17th ACM Conference on Computer and Communications Security (CCS'10). pp. 260–269. ACM Press (Oct 2010). https://doi.org/10.1145/1866307.1866337
4. Busi, M., Focardi, R., Gul, S., Luccio, F.L.: Automated Analysis of Key Management Policies. In: Joint National Conference on Cybersecurity (ITASEC & SERICS 2025). CEUR (2025)
5. Busi, M., Focardi, R., Luccio, F.: Online repository for "Strands Rocq: Why is a Security Protocol Correct, Mechanically?" (2025), https://github.com/strandsrocq/strandsrocq
6. Busi, M., Focardi, R., Luccio, F.L.: Strands rocq: Why is a security protocol correct, mechanically? In: Proceedings of the 38th IEEE Computer Security Foundations Symposium (CSF 2025). pp. 1–14. IEEE, Santa Cruz, CA, USA (Jun 2025). https://doi.org/10.1109/CSF64896.2025.00022
7. Centenaro, M., Focardi, R., Luccio, F.: Type-Based Analysis of Key Management in PKCS#11 Cryptographic Devices. Journal of Computer Security **21**(6), 971–1007 (2013). https://doi.org/10.3233/JCS-130479
8. Clulow, J.: On the Security of PKCS #11. In: Walter, C.D., Koç, Ç.K., Paar, C. (eds.) Cryptographic Hardware and Embedded Systems - CHES 2003. pp. 411–425. Springer Berlin Heidelberg, Berlin, Heidelberg (2003)
9. Cortier, V., Delaune, S., Dreier, J., Klein, E.: Automatic Generation of Sources Lemmas in Tamarin: Towards Automatic Proofs of Security Protocols. Journal of Computer Security **30**(4), 573–598 (2022). https://doi.org/10.3233/JCS-210053
10. Dax, A., Künnemann, R., Tangermann, S., Backes, M.: How to Wrap it up - A Formally Verified Proposal for the use of Authenticated Wrapping in PKCS#11. In: 32nd IEEE Computer Security Foundations Symposium (CSF'19). pp. 62–6215 (2019). https://doi.org/10.1109/CSF.2019.00012
11. Delaune, S., Kremer, S., Steel, G.: Formal Analysis of PKCS#11 and Proprietary Extensions. Journal of Computer Security **18**(6), 1211–1245 (Nov 2010). https://doi.org/10.3233/JCS-2009-0394
12. Focardi, R., Luccio, F.L.: Secure Key Management Policies in Strand Spaces. In: Dougherty, D., Meseguer, J., Mödersheim, S.A., Rowe, P.D. (eds.) Protocols, Strands, and Logic – Essays Dedicated to Joshua Guttman on the Occasion of his 66.66th Birthday, LNCS, vol. 13066, pp. 175–197. Springer (2021). https://doi.org/10.1007/978-3-030-91631-2_10
13. Focardi, R., Luccio, F.L.: A Formally Verified Configuration for Hardware Security Modules in the Cloud. In: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS '21). pp. 1–17. ACM (2021). https://doi.org/10.1145/3460120.3484785
14. Focardi, R., Luccio, F.L., Steel, G.: An Introduction to Security API Analysis, pp. 35–65. Springer Berlin Heidelberg (2011). https://doi.org/10.1007/978-3-642-23082-0_2
15. Fröschle, S., Sommer, N.: Concepts and Proofs for Configuring PKCS#11. In: Formal Aspects of Security and Trust - 8th Int. Workshop, (FAST'11),. Revised Selected Papers. LNCS, vol. 7140, pp. 131–147. Springer (2011). https://doi.org/10.1007/978-3-642-29420-4_9

16. González-Burgueño, A., Santiago, S., Escobar, S., Meadows, C., Meseguer, J.: Analysis of the PKCS#11 API Using the Maude-NPA Tool. In: Chen, L., Matsuo, S. (eds.) Security Standardisation Research. pp. 86–106. Springer International Publishing (2015)
17. Krüger, S., Späth, J., Ali, K., Bodden, E., Mezini, M.: CrySL: An Extensible Approach to Validating the Correct Usage of Cryptographic APIs. IEEE Transactions on Software Engineering (2019). https://doi.org/10.1109/TSE.2019.2948910
18. Künnemann, R.: Automated Backward Analysis of PKCS#11 v2.20. In: Principles of Security and Trust - 4th International Conference (POST'15). LNCS, vol. 9036, pp. 219–238. Springer (2015). https://doi.org/10.1007/978-3-662-46666-7_12
19. Meier, S., Schmidt, B., Cremers, C., Basin, D.: The TAMARIN Prover for the Symbolic Analysis of Security Protocols. In: Sharygina, N., Veith, H. (eds.) Computer Aided Verification. pp. 696–701. Springer Berlin Heidelberg, Berlin, Heidelberg (2013)
20. OASIS PKCS 11 Technical Committee: PKCS #11 Specification Version 3.1. https://docs.oasis-open.org/pkcs11/pkcs11-spec/v3.1/cs01/pkcs11-spec-v3.1-cs01.pdf (2022), committee Specification 01, OASIS, August 11, 2022
21. Schneier, B.: Applied Cryptography: Protocols, Algorithms, and Source Code in C. John Wiley & Sons, Inc., 2 edn. (1996)
22. Steel, G.: Proposal: Authenticated Attributes for Key Wrap in PKCS#11. https://groups.oasis-open.org/communities/community-home/digestviewer/viewthread?MessageKey=D1EDD2AA-AD98-434A-AD42-91B7E7FBDE82 (2014)
23. Thayer, F.J., Herzog, J.C., Guttman, J.D.: Strand Spaces: Why is a Security Protocol Correct? In: Security and Privacy - 1998 IEEE Symposium on Security and Privacy, Oakland, CA, USA, May 3-6, 1998, Proceedings. pp. 160–171 (1998), https://doi.org/10.1109/SECPRI.1998.674832
24. Wickert, A.K., Baumgärtner, L., Schlichtig, M., Narasimhan, K., Mezini, M.: To fix or not to fix: a critical study of crypto-misuses in the wild. In: 2022 IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom). pp. 315–322. IEEE (2022)